

# P201

## Question #1:

### • Steps:

#### ① initialize counter

for i in the length of current state

if (letter of current state) doesn't equal (letter of goal state)

counter increases

counter is returned

② It lower-bounds the minimum number of actions still needed, since each action can fix at most one differing position

③ No single action can fix more than one position, so you can never need fewer actions than differing positions

### • Scrabble:

① Sum of scrabble [goal[i]] over all differing positions

② It lower-bounds the cost of the specific letter insertions that must eventually happen at each differing position

③ whichever path you take, some action must eventually place goal[i] at position i - and that action's cost is fixed at scrabble [goal[i]] regardless of when it happens or what other detours occur, so summing those per position costs can't exceed the truth

### • Frequency

#### ① initialize count

for i in length of current state

if (letter of current state) doesn't equal (letter of goal state)

counter increases

if counter doesn't equal 0:

Counter = Counter amount + rarity value of goal word

return Counter

② It lower-bounds the total cost of the remaining actions and uses 1 for the unknown intermediate value plus the cost of the known end value

③ Every word costs at least one (but can cost more) then we add the rarity score of the known goal

## Question 2:

| Word pair   | Cost measure | UCS cost    | UCS nodes expanded | A* cost     | A* nodes expanded | Reduction |
|-------------|--------------|-------------|--------------------|-------------|-------------------|-----------|
| WHY → ASK   | Steps        | 5           | 773                | 5           | 73                | 90.56%    |
| WHY → ASK   | Scrabble     | 9           | 765                | 9           | 13                | 98.30%    |
| WHY → ASK   | Frequency    | 38.85540652 | 808                | 38.85540652 | 634               | 21.53%    |
| BEAR → DUCK | Steps        | 4           | 691                | 4           | 6                 | 99.13%    |
| BEAR → DUCK | Scrabble     | 11          | 2981               | 11          | 6                 | 99.79%    |
| BEAR → DUCK | Frequency    | 38.2402284  | 1084               | 38.2402284  | 302               | 72.14%    |

1) Yes they did, there were no discrepancies

2)

- Why → ASK : Scrabble (98.30%)
- Bear → Duck : Scrabble (99.79%)

3)

- Why → ASK : Frequency (21.53%)
- Bear → Duck : Frequency (72.14%)

4) Steps and Scrabble perform better than Frequency because they give feedback for every mismatched position, letting them look at the distance from the goal. Frequency only computes the exact cost for the final action and assigns a lower value of 0 to every other position. For Bear → Duck, where the optimal path has no detours (path length = letter distance), this flaw stays close to the true cost. Frequency can't account for those extra steps because it doesn't know the true cost of the additional words. Scrabble's reduction rate is higher because the cost of each position is the actual letter needed rather than a singular one value like steps. This allows A\* to distinguish between cheap and expensive moves a lot earlier. Steps can't determine that because each action is the same cost.

### Question 3

#### • Steps:

① Yes, To find the cost of steps, every action costs 1 so the optimal cost going back and forth would be the same.

→ Reversible + Symmetric Cost

#### • Scrabble:

① No, Scrabble looks at the value of the different letters so going from A → B doesn't cost the same as going from B → A

for example: dog → cog (cost: 3)

Cog → dog (cost: 2)

the cost is different.

→ Reversible + Not Symmetric Cost

#### • Frequency:

① No, frequency looks at the rarity of the words in between. The words going from dog → cat aren't the same words going from cat → dog

therefore the optimal cost will be different.

→ Reversible + Not Symmetric Cost

→ using a script I wrote:

The cost to go from "the" → "she" = 3.0556157 3.0556157 ≠ 1.016226

The cost to go from "she" → "the" = 1.016226

## Question 4

① Gemini 3.6 Thinking,

Prompt: Prompt given in the assignment

$$h(n) = \begin{cases} 0 & \text{if } n = g \\ h_0(n) + \text{rarity}(g) + (h_0(n) - 1)R_{\min} & \text{if } n \neq g \end{cases}$$

②

```
def rarity_aware_heuristic(state, goal, dictionary_rarity, R_min):
    if state == goal:
        return 0

    # Hamming distance baseline
    h0 = sum(1 for a, b in zip(state, goal) if a != b)

    # Mandatory cost: baseline step cost + goal rarity + minimum intermediate rarity
    return h0 + dictionary_rarity[goal] + (h0 - 1) * R_min
```

③ I think it is admissible

$n = \text{dog}$ , assume rarity 3

$g = \text{cat}$ , assume rarity 4

assume cheapest rarity = 0

$h_0(\text{dog}, \text{cat}) = 3 \Rightarrow$  all three letters need to change

$$h(\text{dog}) = 3 + 4 + (3 - 1)(0) = 7$$

- path

$$\text{dog} : \text{cost} = 1 + \text{rarity}(\text{dog}) \geq 0 = 1$$

$$\text{don} : \text{cost} = 1 + \text{rarity}(\text{don}) \geq 0 = 1$$

$$\text{can} : \text{cost} = 1 + \text{rarity}(\text{can}) \geq 0 = 1$$

$$\text{cat} : \text{cost} = 1 + \text{rarity}(\text{cat}) = 4$$

$$1 + 1 + 1 + 4 = 7$$

$\Rightarrow$  even if the rarity is 0 the cost of each word will still be at least 1

$$= (h_0(n) - 1) \times (1 + R_{\min}) + (1 + \text{rarity}(g))$$

$$= h_0(n) - 1 + (h_0(n) - 1)R_{\min} + 1 + \text{rarity}(g)$$

$$= h_0(n) + (h_0(n) - 1) \times R_{\min} + \text{rarity}(g)$$

this compared to the proposed formula of the AI which is  $h_0(n) + \text{rarity}(g) + (h_0(n) - 1) \times R_{\min}$  is the same, just ordered differently

5.)

| Word       | Cost measure | UCS Cost | UCS nodes Expanded | no Cost | no Nodes Expanded | Proposed Cost | Proposed Path |
|------------|--------------|----------|--------------------|---------|-------------------|---------------|---------------|
| dog<br>cat | frequency    | 23.02223 | 229                | 23.0222 | 29                | 23.022        | 129           |

• The evidence does support using the proposed heuristic because the Cost ends up being the same. Although the Nodes Expanded is greater than the given A\*